

# Accelerating Multigrid Prolongation via bipartite graph attention networks<sup>1</sup>

Mitchell Scott<sup>1</sup>, Socratis Petrides<sup>2</sup>, Rui Peng Li<sup>2</sup>

<sup>1</sup> Department of Mathematics, Emory University,

<sup>2</sup> Center for Applied Scientific Computing, LLNL

September 25, 2026

---

<sup>1</sup>This work was partially supported by Lawrence Livermore National Laboratory under grants CASC LDRD 50962/24ERD033\* and 38591/DSSI.Students.



# Solving $Au = f$ when $A$ is large and sparse

**Problem.** An elliptic PDE discretized with finite elements (here: MFEM) gives  $Au = f$ , with  $A$  sparse and SPD:  $n$  unknowns,  $\approx 7$  nonzeros per row.

**Direct** (sparse Cholesky): fill-in.

- $O(n^{3/2})$  work in 2D [George 1973]
- $O(n^2)$  in 3D.

## 2D Poisson on these meshes

| $n$      | GMRES alone             | + multigrid      |
|----------|-------------------------|------------------|
| 961      | 116                     | 9.5              |
| 2209     | 174                     | 9.5              |
| 3969     | 227                     | 9.8              |
| 6241     | 284                     | 10.0             |
| $\vdots$ | $\vdots$                | $\vdots$         |
| $n$      | $\mathcal{O}(\sqrt{n})$ | $\mathcal{O}(1)$ |



# Solving $Au = f$ when $A$ is large and sparse

**Problem.** An elliptic PDE discretized with finite elements (here: MFEM) gives  $Au = f$ , with  $A$  sparse and SPD:  $n$  unknowns,  $\approx 7$  nonzeros per row.

**Direct** (sparse Cholesky): fill-in.

- $O(n^{3/2})$  work in 2D [George 1973]
- $O(n^2)$  in 3D.

**Iterative** (CG, GMRES):

- each iteration is a sparse mat-vec
- $\kappa(A) = O(h^{-2})$ , with small  $h$

## 2D Poisson on these meshes

| $n$      | GMRES alone             | + multigrid      |
|----------|-------------------------|------------------|
| 961      | 116                     | 9.5              |
| 2209     | 174                     | 9.5              |
| 3969     | 227                     | 9.8              |
| 6241     | 284                     | 10.0             |
| $\vdots$ | $\vdots$                | $\vdots$         |
| $n$      | $\mathcal{O}(\sqrt{n})$ | $\mathcal{O}(1)$ |



# Solving $Au = f$ when $A$ is large and sparse

**Problem.** An elliptic PDE discretized with finite elements (here: MFEM) gives  $Au = f$ , with  $A$  sparse and SPD:  $n$  unknowns,  $\approx 7$  nonzeros per row.

**Direct** (sparse Cholesky): fill-in.

- $O(n^{3/2})$  work in 2D [George 1973]
- $O(n^2)$  in 3D.

**Iterative** (CG, GMRES):

- each iteration is a sparse mat-vec
- $\kappa(A) = O(h^{-2})$ , with small  $h$

**Question:** How do we improve an approximate solution at  $O(n)$  cost per step, in a number of steps that does not grow with  $n$ ?

## 2D Poisson on these meshes

| $n$      | GMRES alone   | + multigrid |
|----------|---------------|-------------|
| 961      | 116           | 9.5         |
| 2209     | 174           | 9.5         |
| 3969     | 227           | 9.8         |
| 6241     | 284           | 10.0        |
| $\vdots$ | $\vdots$      | $\vdots$    |
| $n$      | $O(\sqrt{n})$ | $O(1)$      |

# Multigrid: smooth the error, then correct it on a coarse grid

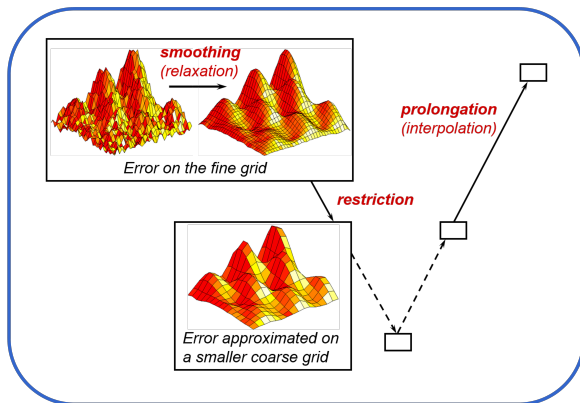


Figure: hypre team, LLNL

Everything hinges on  $P$ . AMG builds it from the matrix alone; this talk learns its values.

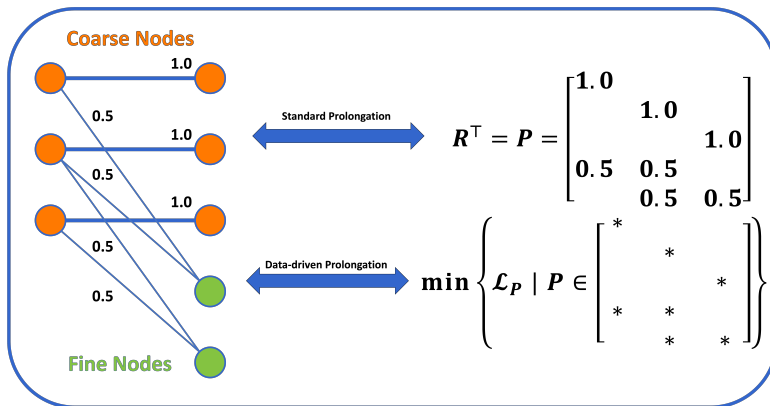
```

1: function V-CYCLE( $A_\ell, b, x$ )
2:   if  $\ell = L$  then return  $A_L^{-1}b$ 
3:    $x \leftarrow x + S_{\text{pre}}^{-1}(b - A_\ell x)$ 
4:    $r_c \leftarrow P_\ell^T(b - A_\ell x)$ 
5:    $e_c \leftarrow \text{V-CYCLE}(A_{\ell+1}, r_c, 0)$ 
6:    $x \leftarrow x + P_\ell e_c$ 
7:    $x \leftarrow x + S_{\text{post}}^{-1}(b - A_\ell x)$ 
8:   return  $x$ 
9: end function
  
```

▷ smoothing  
 ▷ restriction  
 ▷ coarse grid  
 ▷ prolongation  
 ▷ smoothing

Setup, once:  $A_{\ell+1} = P_\ell^T A_\ell P_\ell$ .  
 Smoothers  $S$ : Gauss-Seidel.

# Standard vs. data-driven prolongation

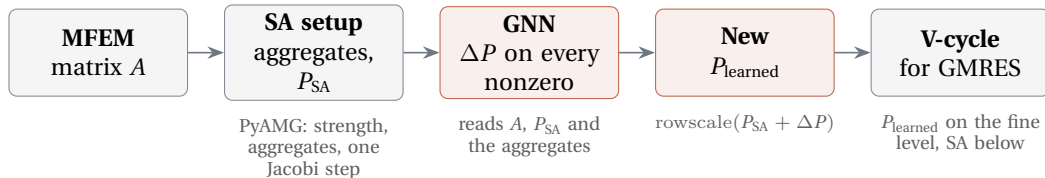


Standard prolongation fixes the weights (here 1 and  $\frac{1}{2}$ ).

Data-driven prolongation keeps the edges and **learns the weights** by minimising a loss  $\mathcal{L}_P$ .



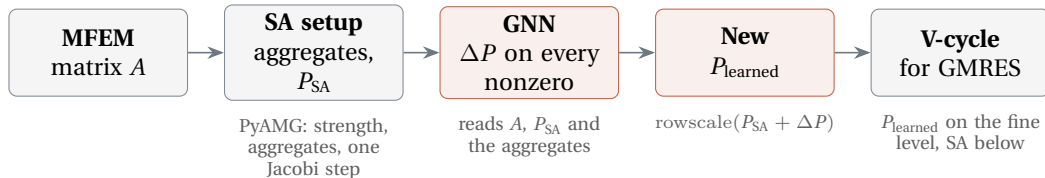
# Our method: a GNN refines smoothed aggregation's $P$



SA [Vaněk et al. 1996]. Closest prior work: a GNN setting  $P$ 's values on a classical-AMG pattern [Luz et al. 2020].



# Our method: a GNN refines smoothed aggregation's $P$



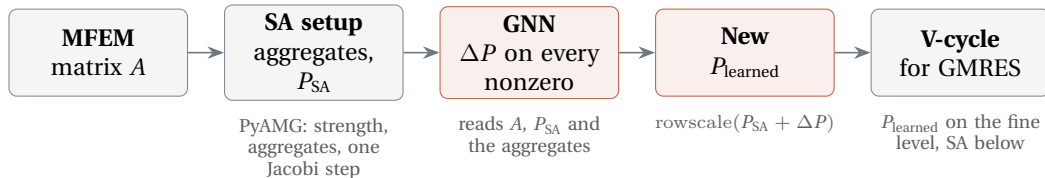
**Same aggregates, same nonzeros**  
**Starts at Standard Aggregation (SA)**  
**Same near-nullspace action**

only the values of  $P$  change  
untrained, it returns  $P_{SA}$  exactly  
rows rescaled so  $P_{\text{learned}} B_c = P_{SA} B_c$

SA [Vaněk et al. 1996]. Closest prior work: a GNN setting  $P$ 's values on a classical-AMG pattern [Luz et al. 2020].



# Our method: a GNN refines smoothed aggregation's $P$



**Same aggregates, same nonzeros**

only the values of  $P$  change

**Starts at Standard Aggregation (SA)**

untrained, it returns  $P_{SA}$  exactly

**Same near-nullspace action**

rows rescaled so  $P_{\text{learned}} B_c = P_{SA} B_c$

**Message passing:** 3 layers on the graph of  $A$ ; 2 rounds dofs  $\rightarrow$  aggregate  $\rightarrow$  dofs; then an MLP on each edge  $(i, J)$  of  $P \leftarrow 34,145$  parameters, one set for every mesh size.

SA [Vaněk et al. 1996]. Closest prior work: a GNN setting  $P$ 's values on a classical-AMG pattern [Luz et al. 2020].



# The loss targets the solver's two-grid convergence factor

Supervised:  $\|P - P_{\text{true}}\|$  needs a target  $P$ . **Unsupervised:**  $\mathcal{L}_P = \|E(P)\|$  needs only  $A$ ,

$$E(P) = I - M_{\text{TG}}^{-1}A = (I - S_{\text{post}}^{-1}A)(I - PA_c^{-1}P^T A)(I - S_{\text{pre}}^{-1}A), \quad A_c = P^T AP$$



# The loss targets the solver's two-grid convergence factor

Supervised:  $\|P - P_{\text{true}}\|$  needs a target  $P$ . **Unsupervised:**  $\mathcal{L}_P = \|E(P)\|$  needs only  $A$ ,

$$E(P) = I - M_{\text{TG}}^{-1}A = (I - S_{\text{post}}^{-1}A)(I - PA_c^{-1}P^{\text{T}}A)(I - S_{\text{pre}}^{-1}A), \quad A_c = P^{\text{T}}AP$$

**True target**  $\rho(E) = \|E\|_A$ : exact, but an eigensolve per matrix at every step, and not smooth where eigenvalues cross.

**Frobenius**  $\|E\|_F$ : cheap and smooth, but typically  $\sim 5\times$  too big. [Luz et al. 2020]

**Gelfand**  $\|E^k\|_F^{1/k} \rightarrow \rho(E)$ : smooth and tight, typically  $\sim 10\%$  above at  $k = 8$ . [Katrutsa et al. 2020]  
**This work.**



# The loss targets the solver's two-grid convergence factor

Supervised:  $\|P - P_{\text{true}}\|$  needs a target  $P$ . **Unsupervised:**  $\mathcal{L}_P = \|E(P)\|$  needs only  $A$ ,

$$E(P) = I - M_{\text{TG}}^{-1}A = (I - S_{\text{post}}^{-1}A)(I - PA_c^{-1}P^T A)(I - S_{\text{pre}}^{-1}A), \quad A_c = P^T A P$$

**True target**  $\rho(E) = \|E\|_A$ : exact, but an eigensolve per matrix at every step, and not smooth where eigenvalues cross.

**Frobenius**  $\|E\|_F$ : cheap and smooth, but typically  $\sim 5\times$  too big. [Luz et al. 2020]

**Gelfand**  $\|E^k\|_F^{1/k} \rightarrow \rho(E)$ : smooth and tight, typically  $\sim 10\%$  above at  $k = 8$ . [Katrutsa et al. 2020]  
**This work.**

| $k$                         | 1           | 2    | 4    | 8           | 16   | 32   |
|-----------------------------|-------------|------|------|-------------|------|------|
| $\ E^k\ _F^{1/k} / \rho(E)$ | <b>5.13</b> | 1.89 | 1.27 | <b>1.10</b> | 1.04 | 1.02 |

$$\mathcal{L} = \log \|E(P_{\text{learned}})^8\|_F^{1/8} - \log \|E(P_{\text{SA}})^8\|_F^{1/8}$$

Negative = beats SA on this bound.



# Rotated Anisotropic Diffusion: coarsening misreads the operator

$$-\nabla \cdot (K \nabla u) = f \text{ on } (0, 1)^2, \quad K = R(\theta)^T \begin{pmatrix} 1 & 0 \\ 0 & \varepsilon \end{pmatrix} R(\theta)$$

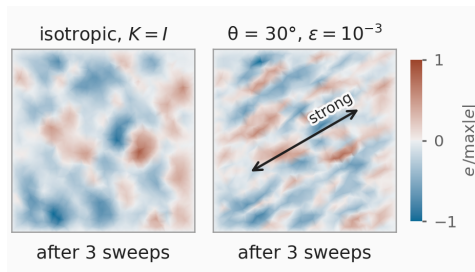
Relaxation smooths the error only along the **strong direction**  $\theta$ , so the coarse grid has to follow it.

AMG must find that direction from the matrix alone

Smoothed aggregation's GMRES iterations grow **18**  $\rightarrow$  **32** as  $n$  grows 961  $\rightarrow$  6241. (symmetric strength)

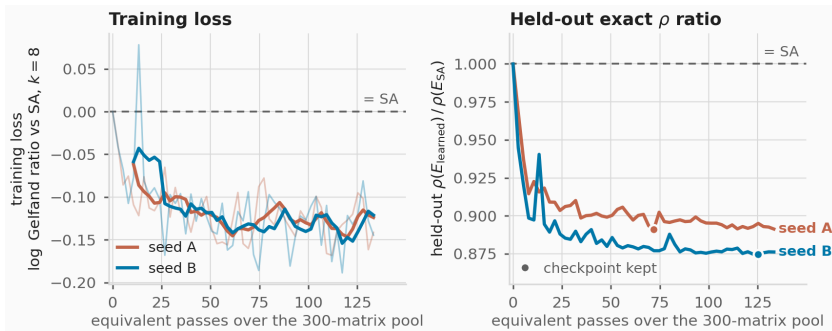
## 24 test problems

6 pairs  $(\theta, \varepsilon)$ ,  $\theta$  from  $20^\circ$  to  $138^\circ$ ,  $\varepsilon$  from  $10^{-4}$  to  $10^{-2}$ , each at  $n = 961, 2209, 3969, 6241$ .



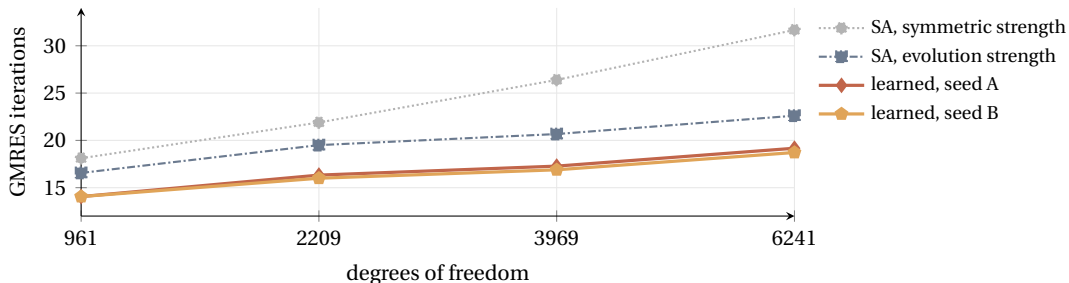


# Training: small matrices, no target operator



|              | $\theta$                     | $\epsilon$                | $n$      |                                |
|--------------|------------------------------|---------------------------|----------|--------------------------------|
| <b>Train</b> | $0^\circ\text{--}180^\circ$  | $10^{-4}\text{--}10^{-1}$ | 100–289  | 300 random draws               |
| <b>Test</b>  | $20^\circ\text{--}138^\circ$ | $10^{-4}\text{--}10^{-2}$ | 961–6241 | 6 fixed pairs $\times$ 4 sizes |

# We require fewer iterations than any standard coarsening tried.



**13–18% fewer GMRES iterations** on average than the best of 41 tuned PyAMG coarsenings, and fewer on **all 24** test problems, for both trained networks.

# Iterations are not yet wall-clock

| Method                                 | Setup        | Iterations | GMRES solve  | Total  |
|----------------------------------------|--------------|------------|--------------|--------|
| SA, symmetric strength                 | <b>52 ms</b> | 32         | 69 ms        | 121 ms |
| SA, evolution strength                 | 103 ms       | 23         | 47 ms        | 150 ms |
| Learned, symmetric-strength aggregates | 165 ms       | 23         | <b>43 ms</b> | 208 ms |
| Learned, evolution-strength aggregates | 226 ms       | <b>20</b>  | <b>40 ms</b> | 266 ms |

*Break-even is amortization over right-hand sides:  $\approx 4$  against symmetric-strength SA; 6–16 against evolution strength, where the solve saves under 20 ms and runs vary. Training (about 10 minutes, once, offline) is not the obstacle.*



# Conclusions and future work

## Conclusions

- 1 On rotated anisotropic diffusion over unstructured MFEM meshes, a prolongator trained *without a target operator* needs **13–18% fewer GMRES iterations** on average than the best of 41 standard coarsenings — on all 24 problems, for both runs, at lower average operator complexity.
- 2 The gain comes from the *values* of the fine-level  $P$ : SA's aggregates, nonzero positions and near-nullspace action are all kept.
- 3 The two-grid convergence factor is a label-free training signal that tracks the solver (Spearman 0.991 against iterations, 7 prolongators on one matrix).

## Future work

- Adaptive sparsity for learned prolongator.
- Other families: jump coefficients, convection, 3D, higher order.
- Cut inference cost until a single solve wins on wall-clock.
- Compare with hypre's BoomerAMG; learn the sparsity pattern too; pair with learned smoothers [Huang, Li & Xi 2023].



# References and acknowledgements

- W. L. Briggs, V. E. Henson, S. F. McCormick. *A Multigrid Tutorial*, 2nd ed. SIAM, 2000.
- A. George. Nested dissection of a regular finite element mesh. *SIAM J. Numer. Anal.* 10 (1973) 345–363.
- P. Vaněk, J. Mandel, M. Brezina. Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems. *Computing* 56 (1996) 179–196.
- L. N. Olson, J. Schroder, R. S. Tuminaro. A new perspective on strength measures in algebraic multigrid. *Numer. Linear Algebra Appl.* 17 (2010) 713–733.
- I. Luz, M. Galun, H. Maron, R. Basri, I. Yavneh. Learning algebraic multigrid using graph neural networks. *ICML 2020*, PMLR 119:6489–6499.
- N. Bell, L. N. Olson, J. Schroder. PyAMG: algebraic multigrid solvers in Python. *J. Open Source Softw.* 7(72) (2022) 4142.
- R. Anderson et al. MFEM: a modular finite element methods library. *Comput. Math. Appl.* 81 (2021) 42–74.
- R. Huang, R. Li, Y. Xi. Learning optimal multigrid smoothers via neural networks. *SIAM J. Sci. Comput.* 45(3) (2023).

## Acknowledgements

This work was partially supported by Lawrence Livermore National Laboratory under grants CASC LDRD 50962/24ERD033\* and 38591/DSSI.Students. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of Lawrence Livermore National Lab, the National Nuclear Security Administration, or the Department of Energy. Prepared by LLNL under Contract DE-AC52-07NA27344.

# Backup: the 41 standard solvers

41 PyAMG solvers, all of which built a multilevel hierarchy and converged at every size:

- 24 classical: C/F splittings RS, RS with second pass, CLJP, CLJPc, PMIS, PMISc  $\times$  classical and direct interpolation  $\times$  thresholds 0.25, 0.5. (PyAMG's Ruge–Stüben solver does not accept MIS, so it drops out.)
- 11 smoothed aggregation: standard and Lloyd aggregation with symmetric strength at thresholds 0, 0.08, 0.25, 0.5; evolution strength at  $\epsilon = 2, 4, 8$ .
- 6 root-node AMG: energy minimisation of degree 1 and 2 on symmetric (0.25, 0.08) and evolution ( $\epsilon = 4$ ) strength.

Identical V(1,1) Gauss–Seidel cycle, pseudo-inverse coarse solve and GMRES for every method; only the coarsening and its interpolation vary.

“Best of 41” is chosen *per problem, after* seeing its iteration count — an oracle over hyperparameters the learned model never gets.

The winner is SA on evolution strength in 19 of the 24 cases; root-node AMG takes 3 and CLJPc 2 (ties broken by lower operator complexity). The SA baselines on the results slide use the best threshold of each kind (symmetric 0.25, evolution  $\epsilon = 2$ ); PyAMG's defaults (0 and 4) average 36–63 and 20–30 iterations across the four sizes.

## Backup: the unsupervised objective tracks the solver

| Prolongator (same aggregates)       | $\rho(M)$     | GMRES its | V-cycle factor | $\text{nnz}(P)$ |
|-------------------------------------|---------------|-----------|----------------|-----------------|
| Tentative $T$ (unsmoothed)          | 0.8982        | 31        | 0.9005         | 958             |
| Jacobi on strength graph, deg. 1    | 0.8647        | 27        | 0.8832         | 1751            |
| Energy-min, degree 1                | 0.8554        | 26        | 0.8798         | 1751            |
| Energy-min, degree 2                | 0.8467        | 24        | 0.8471         | 3040            |
| Jacobi on strength graph, deg. 2    | 0.8449        | 24        | 0.8474         | 3040            |
| Smoothed aggregation (PyAMG)        | 0.7871        | 18        | 0.7820         | 3636            |
| <b>Learned</b> (seed A, fine level) | <b>0.6995</b> | <b>16</b> | <b>0.7045</b>  | 3636            |

Spearman correlation  $\rho(M)$  vs. iterations: **0.991**. The ranking is monotone; the only deviation is a tie, the two degree-2 rows both landing on 24 iterations.

The standalone V-cycle factor — no Krylov acceleration — drops from 0.782 to 0.705, so the gain is multigrid quality, not something GMRES happens to exploit.

# Backup: why $k = 8$ , and why not differentiate $\rho(M)$

- **Smoothness.**  $\rho(M)$  is the modulus of a *single* eigenvalue: not differentiable where the leading eigenvalues cross, and minimising it tends to push them together.
- **A smooth upper bound.**  $\|M^k\|_F^{1/k} \geq \rho(M)$  for every  $k$ , and it is differentiable in  $P$ , so the gradient is usable everywhere.
- **$k = 1$  is not an option.**  $M$  is non-normal:  $\rho(M) = 0.61$  while  $\|M\|_2 = 1.16$  and  $\|M\|_F = 3.13$ .
- Exact  $\rho(M)$  is still computed — under `no_grad`, via `eigvals` — as the validation metric that selects the checkpoint.

## Gelfand surrogate vs. $\rho(M)$

| $k$      | $\ M^k\ _F^{1/k} / \rho(M)$ |
|----------|-----------------------------|
| 1        | 5.13                        |
| 2        | 1.89                        |
| 4        | 1.27                        |
| <b>8</b> | <b>1.10</b>                 |
| 16       | 1.04                        |
| 32       | 1.02                        |

The bias left at  $k = 8$  is near-multiplicative and largely cancels in the log-ratio.



# Backup: What does not work

## Depth, not seed, is the fragile axis

Both seeds improve the *fine* level:  $\rho(M)$  from 0.787 to 0.700 and 0.696 ( $n = 961$ , symmetric-strength aggregates).

Applied recursively to Galerkin coarse operators they come apart: at the first coarse level ( $n = 277$ ) seed A lowers  $\rho(M)$  by 11%, seed B *raises* it by 30%. On every level of the evolution-strength hierarchy, seed B needs 37.7 iterations at  $n = 6241$ ; SA needs 22.6.

Confined to the fine level, as in every headline result, the two seeds' mean iteration counts agree to within half an iteration.

## It is a specialist

On jump-coefficient problems ( $n = 961$ ) the learned P turns SA's 11.5 iterations into 17.0 (seed A) and 23.5 (seed B); a classical C/F splitting needs 8.

One training run on a mix that included them (a wider, 64k-parameter network) collapsed: their  $\rho(M)$  ratio never went below 1, and after  $\sim 1200$  steps the model beat SA on only 1–4% of validation matrices.

Neither failure is visible in validation, which sees only 100–289-dof matrices; only 8 (seed A) and 10 (seed B) of the 300 training matrices are Galerkin coarse operators.



# Backup: hierarchy at $n = 6241$

|                                 | Level sizes             | Op. complexity |
|---------------------------------|-------------------------|----------------|
| SA, evolution strength          | 6241, 2002, 530, 92, 18 | 2.84           |
| Learned (seed A) at level 0     | 6241, 2002, 453, 93, 18 | 2.72           |
| Learned (seed A) at every level | 6241, 2002, 453, 68, 10 | 2.66           |

Capping the network at level 0 does *not* leave the hierarchy below untouched:  $A_1 = P_0^T A P_0$  changes, so the aggregation computed on it changes too. The fine-level coarse grid is identical by construction (2002 either way); everything from level 2 down is not.

